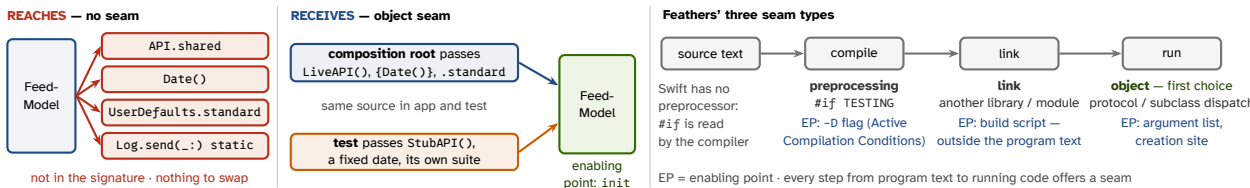
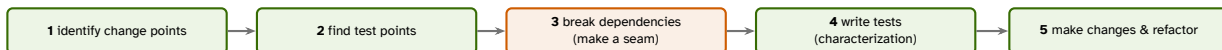


In one line: A **seam** is “a place where you can alter behavior in your program without editing in that place”; every seam has an **enabling point**, “a place where you can make the decision to use one behavior or another” (Michael Feathers, *Working Effectively with Legacy Code*, 2004). In Swift the seam is a **protocol or closure** the type *receives*, and the enabling point is its `init`. Code that *reaches for* `.shared`, `Date()` or `UserDefaults.standard` has none.



Legacy code change algorithm (Feathers)

the dilemma: to change code safely you need tests; to add tests you often must change code ⇒ keep step 3 small and conservative



Seams — the rules

- The source is **the same in production and test**; only the enabling point differs.
- An object **created and used in the same method** is no seam — nothing can choose its type. Pass it in and the **argument list** becomes the enabling point.
- Object seams are “the best choice in object-oriented languages”; preprocessing and link seams are “not as explicit”, their tests hard to maintain — keep them for pervasive dependencies, and make test vs production obvious.
- Two reasons to break a dependency: **sensing** (see values the code computes but hides) and **separation** (stand the code up in a harness at all).
- Fowler (2024): a seam also hosts **probes** and **redirects flow** to new modules when displacing a legacy system.
- Apple (WWDC17): testable code gives **control over inputs, visibility into outputs, no hidden state** — via protocols + parameters, side effects kept apart.

Reaches for → receives → the test passes

reaches for	receives	a test passes
<code>X.shared</code> <code>Date(), Date.now</code> <code>Task.sleep, timers</code>	a protocol (any <code>XClient</code>) <code>now: () -> Date</code> any <code>Clock<Duration></code> (iOS 16) <code>() -> UUID</code> using: inout some <code>RandomNumberGenerator</code>	<code>stub / fake</code> <code>{ Date(timeIntervalSince1970: 0) }</code> <code>TestClock, ImmediateClock (swift-clocks);</code> a clock's now is an <code>Instant</code> , not a <code>Date</code> a counter a seeded generator — Apple: “repeatable ... for testing” <code>UserDefaults(suiteName:)</code> , then <code>removePersistentDomain(forName:)</code>
<code>UUID()</code> <code>..<code>random(in:)</code>, <code>shuffled()</code> <code>UserDefaults</code> <code>.standard</code> <code>FileManager + Documents</code></code>	a base-directory URL	<code>URL.temporaryDirectory + a UUID folder</code>
<code>NotificationCenter</code> <code>.default</code> <code>UIApplication.shared</code>	a <code>NotificationCenter</code>	<code>NotificationCenter()</code> — a private center
<code>Locale, TimeZone</code> <code>.current</code>	a protocol of the members used the value	a spy recording <code>open(...)</code> <code>Locale(identifier:)</code> , <code>TimeZone(secondsFromGMT: 0)</code>

@testable import and the test host

- Tests reach internal only with `@testable import` and a module compiled with testing enabled; never `fileprivate/private` — needing those means “extract a type”, not “make it public”. They may subclass / override non-final public and internal classes (SE-0117).
- Enable Testability** (ENABLE_TESTABILITY) passes `-enable-testing` and `-rdynamic`, drops `-fvisibility=hidden` and strip — “tests running slower”, so Debug only. A Release-built module: “Module ... was not compiled for testing”.
- Test Host** set ⇒ Xcode starts the app and loads the test bundle into it; XCTest waits for `didFinishLaunching` — the composition root runs first. Apple: skip non-essential launch work via a launch argument, or use a host-less target.

Swift 6 concurrency meets test hooks

- SE-0412: every **static** or global stored property is global-actor isolated or an immutable Sendable value (error in Swift 6 mode) — a `static var shared` test hook needs `@MainActor` or `nonisolated(unsafe)`.
- Swift Testing runs tests **in parallel** in one process (task groups); .serialized orders only its own suite — a swapped global still races other suites.
- Parallel-safe ambient seam: a `@TaskLocal` value + a `TestScoping` trait (Swift 6.1) wrapping each test in `$current.withValue(mock) { try await function() }`.
- New app projects in Xcode 26 turn on main-actor mode (SE-0466, Swift 6.2, per module); app types become `@MainActor`, so tests await them or run `@MainActor`.

Traps

- ! **Live-default leak**: one forgotten argument and the test gets the real `LiveFeedAPI` — network in a unit test. Pass every dependency in tests.
- ! A suite does not isolate `register(defaults:)`: the registration and argument domains are shared by every `UserDefaults` instance.
- ! 8 init parameters is an SRP smell, not a DI problem. DI ≠ DIP: injecting a concrete class is DI without inversion.

Legacy code

- “Legacy code is simply **code without tests**” (WELC preface) — not old code. **Edit and Pray** (plan, edit, then poke around — “pretty much the industry standard”) vs **Cover and Modify** (tests as the safety net).
- Four reasons to change: feature · bug · design · resource usage. **Refactoring** changes structure only, **optimising** only resource usage — both hold functionality invariant. Before any change ask: what changes · how will we know it is right · how will we know nothing broke?

Dependency-breaking moves

WELC (examples in Java, C++, C, C#) catalogues **24** conservative edits in ch.25; the Swift-relevant ones:

Parameterize Constructor	<code>init(api: any API = LiveAPI.shared)</code> — every existing caller compiles unchanged
Parameterize Method	one method takes it: <code>isExpired(now: Date = .now)</code>
Extract Interface	a protocol of just what the SUT calls; the class conforms
Extract Implementer	rename the class <code>LiveAPI</code> ; the old name becomes the protocol
Adapt Parameter	a parameter type you cannot fake → your own narrow protocol + an adapter. Swift needs no wrapper: the framework class conforms in an extension (Apple, WWDC18)
Subclass & Override Method (+ Extract & Override Call / Getter)	a test subclass overrides the I/O; needs a non-final class and the method in the class body — extension members cannot be overridden unless <code>@objc</code>
Introduce Static Setter	a singleton gains a setter for tests (see SE-0412, left)
Break Out Method Object	monster method → new type; its locals become fields
Sprout Method / Class	(ch. 6) new code in a new, test-driven unit; old code only calls it
Wrap Method / Class	rename the old method; a new one with its name runs new + old

Characterization tests (golden master)

- “Document your system's **actual** behavior, not check for the behavior you wish your system had” (Feathers) — a system in production “becomes its own specification”. Keep a surprising result pinned until it is *confirmed* a bug.
- Recipe: code in a harness → assert a value you know is wrong → the failure prints the actual one → expect that → repeat.
- Tools: **swift-snapshot-testing** records a missing reference and fails once; `.dump` snapshots any value via `Mirror`; `assertInlineSnapshot` writes into the test file.
- ApprovalTests.Swift**: `.received` vs `.approved` files.

Example — Parameterize Constructor

```
final class FeedModel {
    private let api: any FeedAPI; private let now: () -> Date
    private let defaults: UserDefaults
    init(api: any FeedAPI = LiveFeedAPI.shared, // old callers
         now: @escaping () -> Date = { Date() }, // unchanged
         defaults: UserDefaults = .standard) {
        self.api = api; self.now = now; self.defaults = defaults } }
// test
let suite = "test-\(UUID())"
let d = UserDefaults(suiteName: suite)!
defer { d.removePersistentDomain(forName: suite) }
let sut = FeedModel(api: StubFeedAPI(posts: []),
                    now: { Date(timeIntervalSince1970: 0) }, defaults: d)
```

Designs that make seams cheap

- DI** (name settled by Fowler and IoC authors, 2004): constructor (“type 3”, `PicoContainer`) · setter (type 2, `Spring`) · interface injection (type 1, `Avalon`). A **service locator** is asked: every user depends on it, and its needs show only as locator calls in the body. Fowler: the choice matters less than separating configuration from use; Seemann (2010): an anti-pattern.
- Humble Object** (Meszaros): move logic out of what is hard to test (view, controller) until it holds the minimum of behaviour — MVVM, Passive View.
- Functional core, imperative shell** (Bernhardt, 2012): decisions are pure functions on values; a thin shell does I/O. The core needs no doubles.